

Phase 3 - NumPy & Pandas

Note: Material comes from

- W3Schools: <https://www.w3schools.com/python/pandas/default.asp>
- Geeksforgeeks: <https://www.geeksforgeeks.org/>
- pydata.org: <https://pandas.pydata.org/>
- And some custom stuff

Recall My Background As A Former Software Developer...

... and the fact that I've been out of college for ~25 years

Time to share some wisdom & life lessons!

Warning!!!

To use a programming language and its libraries most effectively, you need to

1. Know the syntax of the language & libraries.
2. Understand the internals of how the language & libraries work.

That's beyond the scope of this course and the Data Science major at Centre.

What Is Done

CSC 170: learned ***some*** syntax, but focus was on concepts of programming languages (conditionals, loops, functions, etc.) & problem-solving - you don't know Python fully and don't know ***any*** of its internals

DSC 270: learn ***some*** NumPy & Pandas syntax, but focus is on data manipulation

To learn the internals of NumPy & Pandas, you need to know the internals of Python first.

No college really teaches that except as an upper level CSC course if at all and the focus is still mostly conceptual rather than internals of Python specifically.

Where Does This Leave Us?

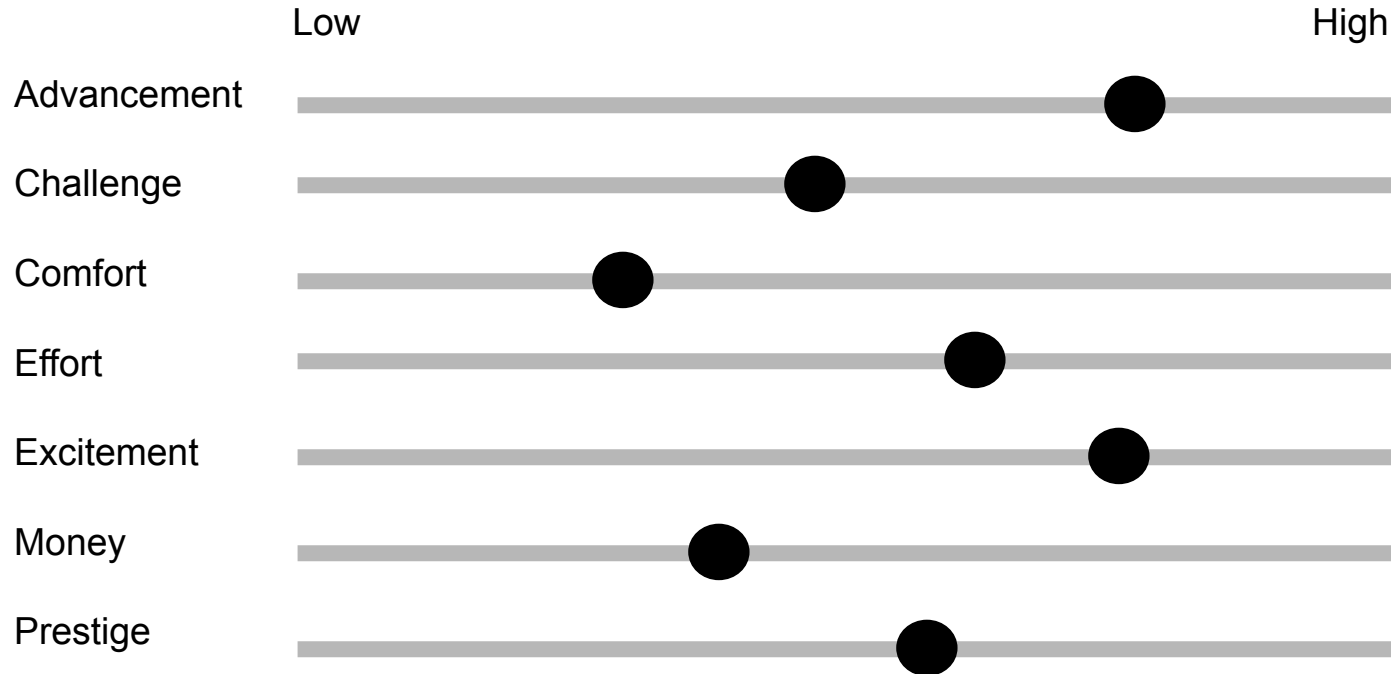
If you were CSC majors:

- You probably still wouldn't care (today) about efficiency & effectiveness at that level.
- You want to know general techniques to solve problems to get a job.
- More learning would take place on the job!

But you're likely all DSC majors.

- You probably don't care (today) about efficiency & effectiveness at that level.
- You want to know general techniques to manipulate data to let you solve problems to get a job.
- More learning will take place on the job!

Everyone Is Different! What Do You Want?



What Does Your Future Hold?

Where you move the sliders will dictate what you need.

There is no right or wrong combination, but they all have consequences.

Don't care about much more than comfort? You need to learn less and be less skilled. But you likely won't move much beyond entry level jobs, won't make as much money or work for prestigious companies or work on projects that are as exciting.

Summary

Beyond scope of the major (and likely your desires) to teach you everything, especially the complete syntax & internals of Python, NumPy, & Pandas.

If you want higher levels of prestige, money, exciting jobs, you may need to independently go learn the syntax and internals of a language.

Sometimes multiple languages!

NumPy & Pandas Overview

NumPy: for working with numerical data in Python (numerical & scientific computing)

Pandas: a Python data analysis library for dealing with structured data (data stored in tables, csv files, Excel sheets, database tables)

NumPy & Pandas Resources

NumPy

- <https://numpy.org/> (installation, documentation, etc.)
- <https://www.w3schools.com/python/numpy/default.asp>

Pandas

- <https://pandas.pydata.org/> (installation, documentation, etc.)
- <https://www.w3schools.com/python/pandas/default.asp>

General

- <https://jakevdp.github.io/PythonDataScienceHandbook/>

NumPy Day 1

NumPy Array Vs. Python List

NumPy array

- Homogeneous
- Faster
- Consumes less memory

NumPy Data Types

Numpy type	C type	Description			
<code>numpy.bool_</code>	<code>bool</code>	Boolean (True or False) stored as a byte	<code>numpy.int_</code>	<code>long</code>	Platform-defined
<code>numpy.byte</code>	<code>signed char</code>	Platform-defined	<code>numpy.ushort</code>	<code>unsigned short</code>	Platform-defined
<code>numpy.ubyte</code>	<code>unsigned char</code>	Platform-defined	<code>numpy.longlong</code>	<code>long long</code>	Platform-defined
<code>numpy.short</code>	<code>short</code>	Platform-defined	<code>numpy.ulonglong</code>	<code>unsigned long long</code>	Platform-defined
<code>numpy.ushort</code>	<code>unsigned short</code>	Platform-defined	<code>numpy.half</code> / <code>numpy.float16</code>		Half precision float: sign bit, 5 bits exponent, 10 bits mantissa
<code>numpy.intc</code>	<code>int</code>	Platform-defined	<code>numpy.single</code>	<code>float</code>	Platform-defined single precision float: typically sign bit, 8 bits exponent, 23 bits mantissa
<code>numpy.uintc</code>	<code>unsigned int</code>	Platform-defined	<code>numpy.double</code>	<code>double</code>	Platform-defined double precision float: typically sign bit, 11 bits exponent, 52 bits mantissa.

NumPy Data Types

<code>numpy.longdouble</code>	long double	Platform-defined extended-precision float
<code>numpy.csingle</code>	float complex	Complex number, represented by two single-precision floats (real and imaginary components)
<code>numpy.cdouble</code>	double complex	Complex number, represented by two double-precision floats (real and imaginary components).
<code>numpy.clongdouble</code>	long double complex	Complex number, represented by two extended-precision floats (real and imaginary components).

Remember To Import NumPy

```
import numpy as np
```

Making A NumPy Array

make an array from a Python list

```
some_array = np.array([1, 2, 3, 4, 5, 6])
```

make a 2-D array from a Python list

```
some_2d_array = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
```

make an array from a Python tuple

```
some_array = np.array((1, 2, 3, 4, 5, 6))
```

Creating Arrays From Scratch

Create a length-10 integer array filled with zeros

`np.zeros(10, dtype=int)`  `array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0])`

Create a 3x5 floating-point array filled with ones

`np.ones((3, 5), dtype=float)`  `array([[1., 1., 1., 1., 1.],
[1., 1., 1., 1., 1.],
[1., 1., 1., 1., 1.]])`

Create a 3x5 array filled with 3.14

`np.full((3, 5), 3.14)`  `array([[3.14, 3.14, 3.14, 3.14, 3.14],
[3.14, 3.14, 3.14, 3.14, 3.14],
[3.14, 3.14, 3.14, 3.14, 3.14]])`

Creating Arrays From Scratch

Create an array filled with a linear sequence

Starting at 0, ending at 20, stepping by 2

(this is similar to the built-in range() function)

`np.arange(0, 20, 2)`  `array([ 0, 2, 4, 6, 8, 10, 12, 14, 16, 18])`

Create an array of five values evenly spaced between 0 and 1

`np.linspace(0, 1, 5)`  `array([ 0. , 0.25, 0.5 , 0.75, 1. ])`

Creating Arrays From Scratch

Create a 3x3 array of uniformly distributed

random values between 0 and 1

`np.random.random((3, 3))`



```
array([[ 0.99844933,  0.52183819,  0.22421193],  
       [ 0.08007488,  0.45429293,  0.20941444],  
       [ 0.14360941,  0.96910973,  0.946117  ]])
```

Create a 3x3 array of normally distributed random values

with mean 0 and standard deviation 1

`np.random.normal(0, 1, (3, 3))`



```
array([[ 1.51772646,  0.39614948, -0.10634696],  
       [ 0.25671348,  0.00732722,  0.37783601],  
       [ 0.68446945,  0.15926039, -0.70744073]])
```

Creating Arrays From Scratch

Create a 3x3 array of random integers in the interval [0, 10)

`np.random.randint(0, 10, (3, 3))`  `array([[2, 3, 4],
 [5, 7, 8],
 [0, 5, 0]])`

Create a 3x3 identity matrix

`np.eye(3)`  `array([[1., 0., 0.],
 [0., 1., 0.],
 [0., 0., 1.]])`

NumPy Array Vocabulary

ndarray: shorthand for N-dimensional array

- 0-D (called a scalar, each value of an array is a 0-D array)
- 1-D (one dimensional array)
- 2-D (two dimensional array)

vector: an array with a single dimension

matrix: an array with a two dimensions

tensor: an array with a 3 or more dimensions

axes: the number of dimensions of an array so an n-dimensional array has n axes

How Many Dimensions?

Given an array, `some_array`, can find number of dimensions with

`some_array.ndim`

Array Indexing

For 1-D arrays, just like list indexing in Python

`some_array[0]` gets the first item in `some_array`

For 2-D (and higher arrays), bracket notation is slightly different!

`some_array[1, 4]` gets 5th element on second row

`some_list[1][4]` gets 5th element on second row

Array Indexing

Negative indexing works, as in Python lists

`some_array[1, -1]` gets last element on second row

Array Slicing

Gets elements from an array.

Step indicates take every stepth element

Uses either

[start:end] or

[start:end:step]

Includes start, but **excludes** end!

```
$ python3 test.py
[ 1  2  3  4  5  6  7  8  9 10]
[3 4 5 6]
[3 5]
```

```
1 import numpy as np
2
3 l = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
4 a = np.array(l)
5
6 # print the array
7 print(a)
8
9 # print a slice of the array
10 print(a[2:6])
11
12 # print another slice of the array
13 print(a[2:6:2])
14
```

More Array Slicing

Can leave start and/or end blank

If leave start blank, slice starts at beginning

If leave end blank, slice goes to end (and includes end)

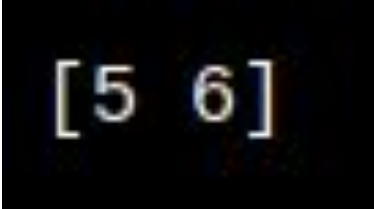
```
[ 5  6  7  8  9 10]
[1 2 3 4]
[ 1  2  3  4  5  6  7  8  9 10]
```

```
15 | print(a[4:])
16 | print(a[:4])
17 | print(a[:])
```

Even More Array Slicing

Can use negative numbers when slicing

```
import numpy as np  
arr = np.array([1, 2, 3, 4, 5, 6, 7])  
print(arr[-3:-1])
```



[5 6]

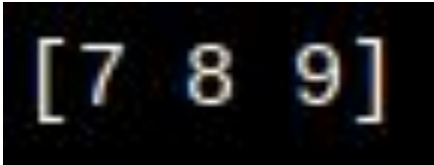
Slicing 2-D Arrays

From the second element, slice elements from index 1 to index 4 (not included):

```
import numpy as np
```

```
arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])
```

```
print(arr[1, 1:4])
```



```
[7 8 9]
```

Slicing 2-D Arrays

From both elements, return index 2:

```
import numpy as np

arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])

print(arr[0:2, 2])
```

[3 8]

Slicing 2-D Arrays

From both elements, slice index 1 to index 4 (not included), this will return a 2-D array:

```
import numpy as np

arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])

print(arr[0:2, 1:4])
```

```
[[2 3 4]
 [7 8 9]]
```

NumPy Day 1 Lab Time!

Lab linked to Moodle

NumPy Day 2

Copy vs View

Writing on a photocopy (copy) vs writing on the original (view)

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
x = arr.copy()
arr[0] = 42

print(arr)
print(x)
```

```
[42  2  3  4  5]
[1  2  3  4  5]
```

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
x = arr.view()
arr[0] = 42

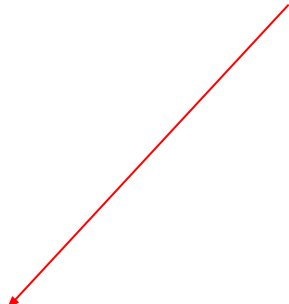
print(arr)
print(x)
```

```
[42  2  3  4  5]
[42  2  3  4  5]
```

Changing Slices

Slicing is VIEWING, NOT copying!

Changing the slice changes the array!



```
toth@toth-Latitude-5520 ~/Desktop
$ python3 test.py
Array: [ 1  2  3  4  5  6  7  8  9 10]
Slice: [3 4 5 6]
Slice: [ 3  4 66  6]
Array: [ 1  2  3  4 66  6  7  8  9 10]
```

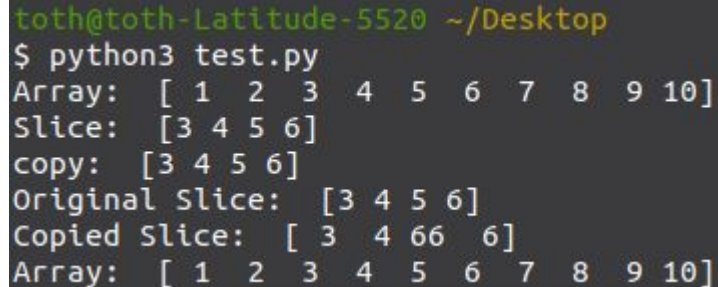
```
1 import numpy as np
2
3 # make an array
4 a = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
5
6 # print the array
7 print("Array: ", a)
8
9 # Take a slice of the array
10 a_slice = a[2:6]
11
12 # print the slice
13 print("Slice: ", a_slice)
14
15 # change the slice
16 a_slice[2] = 66
17
18 # print the slice
19 print("Slice: ", a_slice)
20
21 # print the array
22 print("Array: ", a)
23
```

How Do I Efficiently Use Slices Without Wrecking My Array?

Could copy array & take a slice from that.

(inefficient for big arrays)

Copy the slice!



A terminal window showing the execution of a Python script. The output displays the original array, a slice, a copy of the slice, and the original array after the slice is modified. A red arrow points from the text 'Copy the slice!' to the 'copy:' line in the output.

```
toth@toth-Latitude-5520 ~/Desktop
$ python3 test.py
Array: [ 1  2  3  4  5  6  7  8  9 10]
Slice: [3 4 5 6]
copy: [3 4 5 6]
Original Slice: [3 4 5 6]
Copied Slice: [ 3  4 66  6]
Array: [ 1  2  3  4  5  6  7  8  9 10]
```

```
1 import numpy as np
2
3 # make an array
4 a = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
5
6 # print the array
7 print("Array: ", a)
8
9 # Take a slice of the array
10 a_slice = a[2:6]
11
12 # print the slice
13 print("Slice: ", a_slice)
14
15 # copy the slice
16 b_slice = a_slice.copy()
17
18 # print the copied slice
19 print("copy: ", b_slice)
20
21 # change the slice
22 b_slice[2] = 66
23
24 # print the slices
25 print("Original Slice: ", a_slice)
26 print("Copied Slice: ", b_slice)
27
28 # print the array
29 print("Array: ", a)
30
```

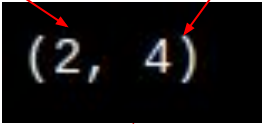
Array Shape

NumPy arrays have **shape** attribute - returns tuple w/ each index showing the number of elements for that dimension

```
import numpy as np
arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])
print(arr.shape)
```

2 elements in
1st dimension

4 elements in 2nd dimension



(2, 4)

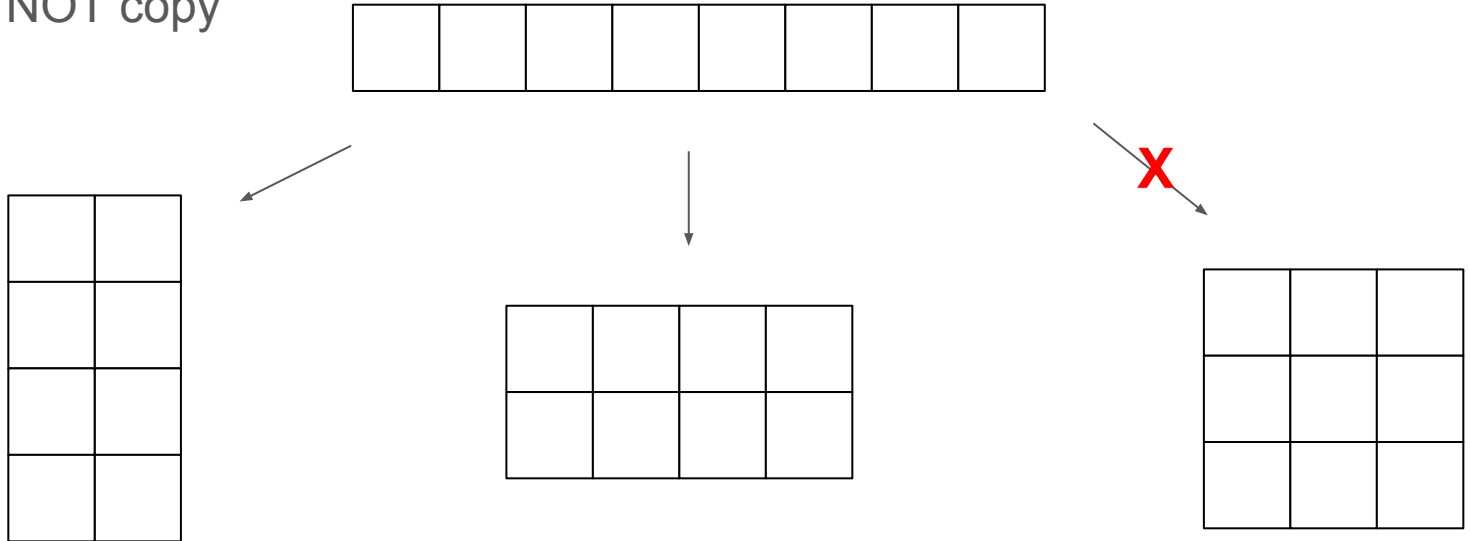
Num items in this tuple indicate
number of dimensions, thus 2

Reshaping Arrays

Add or remove dimensions

Must keep number of elements same in source and destination shape

Returns view, NOT copy



Reshaping Arrays: 1D to 2D

```
import numpy as np  
  
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])  
  
print(arr)  
  
print("\n\n")  
  
newarr = arr.reshape(4, 3)  
  
print(newarr)
```

```
[ 1  2  3  4  5  6  7  8  9 10 11 12]
```

```
[[ 1  2  3]  
 [ 4  5  6]  
 [ 7  8  9]  
 [10 11 12]]
```

Reshaping Arrays: 1D to 3D

Convert the following 1-D array with 12 elements into a 3-D array.

The outermost dimension will have 2 arrays that contains 3 arrays, each with 2 elements:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])

newarr = arr.reshape(2, 3, 2)

print(newarr)
```

```
[[[ 1  2]
   [ 3  4]
   [ 5  6]]

 [[ 7  8]
   [ 9 10]
  [11 12]]]
```

Reshaping Arrays - Unknown Dimensions

When reshaping, can leave 1 dimension “unknown” (specified by -1)

NumPy will fill that in for you at runtime

Convert 1D array with 8 elements to 3D array with 2x2 elements:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])

newarr = arr.reshape(2, 2, -1)

print(newarr)
```

```
[[[1 2]
   [3 4]]

 [[5 6]
   [7 8]]]
```

Reshaping Arrays - Flattening An Array

Convert the array into a 1D array:

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

newarr = arr.reshape(-1)

print(newarr)
```

[1 2 3 4 5 6]

Reshaping Arrays - Other Functions

`flatten`

`ravel`

`rot90`

`flip`

`fliplr`

`flipud`

Iterating Over Arrays

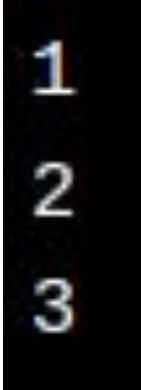
Go through elements 1 by 1

For 1D array

```
import numpy as np

arr = np.array([1, 2, 3])

for x in arr:
    print(x)
```

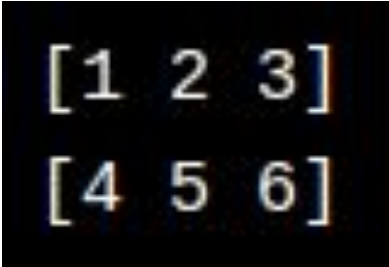


1
2
3

Iterating Over Arrays

For 2D array

```
import numpy as np  
  
arr = np.array([[1, 2, 3], [4, 5, 6]])  
  
for x in arr:  
    print(x)
```



1	2	3
4	5	6

Iterating Over Arrays

For 2D array, getting all scalars

```
import numpy as np
```

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
```

```
for x in arr:  
    for y in x:  
        print(y)
```

1
2
3
4
5
6

Iterating Over Arrays

For 3D array, getting all scalars, nest another loop!

```
import numpy as np

arr = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])

for x in arr:
    for y in x:
        for z in y:
            print(z)
```

1
2
3
4
5
6
7
8
9
10
11
12

Iterating Over Arrays: nditer

Iterate through the following 3-D array:

```
import numpy as np

arr = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])

for x in np.nditer(arr):
    print(x)
```

1

2

3

4

5

6

7

8

Iterating Over Arrays: ndenumerate To See Index Of Items

```
import numpy as np
```

```
arr = np.array([1, 2, 3])
```

```
for idx, x in np.ndenumerate(arr):  
    print(idx, x)
```

```
(0,) 1  
(1,) 2  
(2,) 3
```

```
(0, 0) 1  
(0, 1) 2  
(0, 2) 3  
(0, 3) 4  
(1, 0) 5  
(1, 1) 6  
(1, 2) 7  
(1, 3) 8
```

```
import numpy as np
```

```
arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])
```

```
for idx, x in np.ndenumerate(arr):  
    print(idx, x)
```

Joining Arrays

Combine the contents of multiple arrays into a single array

Pass sequence of arrays to join to the **concatenate()** function, along with the axis.
If axis is not explicitly passed, it is taken as 0.

```
import numpy as np

arr1 = np.array([1, 2, 3])

arr2 = np.array([4, 5, 6])

arr = np.concatenate((arr1, arr2))

print(arr)
```



[1 2 3 4 5 6]

Joining Arrays

Join two 2-D arrays along rows (axis=1):

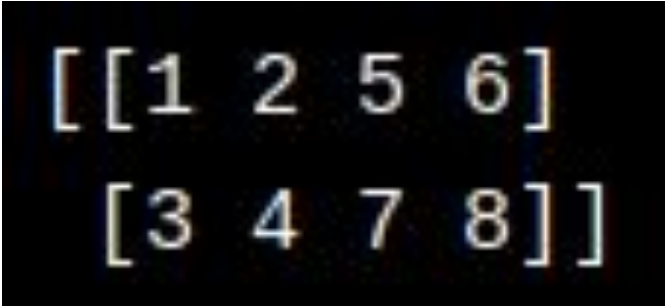
```
import numpy as np

arr1 = np.array([[1, 2], [3, 4]])

arr2 = np.array([[5, 6], [7, 8]])

arr = np.concatenate((arr1, arr2), axis=1)

print(arr)
```



```
[[1 2 5 6]
 [3 4 7 8]]
```

Joining Arrays By Stacking

Stacking like concatenation - difference is stacking is done along a new axis.

Pass arrays we want to join to `stack()` method along with the axis. If axis is not explicitly passed it is taken as 0.

Joining Arrays By Stacking

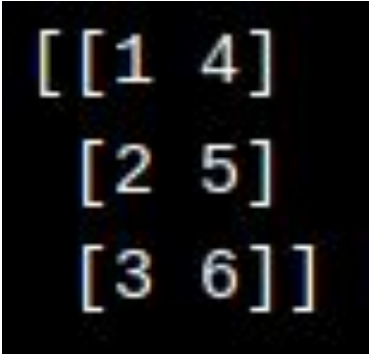
```
import numpy as np

arr1 = np.array([1, 2, 3])

arr2 = np.array([4, 5, 6])

arr = np.stack((arr1, arr2), axis=1)

print(arr)
```



```
[[1 4]
 [2 5]
 [3 6]]
```

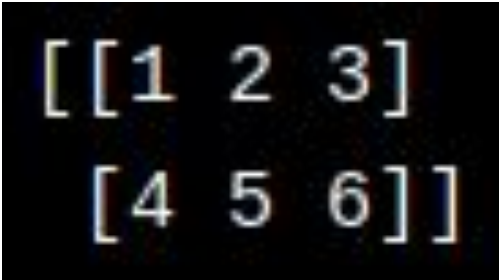
Joining Arrays By Stacking Along Rows: hstack()

```
import numpy as np  
  
arr1 = np.array([1, 2, 3])  
arr2 = np.array([4, 5, 6])  
  
arr = np.hstack((arr1, arr2))  
  
print(arr)
```

```
[1 2 3 4 5 6]
```

Joining Arrays By Stacking Along Columns: vstack()

```
import numpy as np  
  
arr1 = np.array([1, 2, 3])  
arr2 = np.array([4, 5, 6])  
  
arr = np.vstack((arr1, arr2))  
  
print(arr)
```



```
[[1 2 3]  
 [4 5 6]]
```

Joining Arrays By Stacking Along Height/Depth: `dstack()`

Not needed now

Felt like I needed to mention it

Splitting Arrays With `array_split()`

Breaks a single array into multiple arrays

```
import numpy as np  
  
arr = np.array([1, 2, 3, 4, 5, 6])  
  
newarr = np.array_split(arr, 3)  
  
print(newarr)
```

Pass array to split and number of splits (pieces)



```
[array([1, 2]), array([3, 4]), array([5, 6])]
```

Splitting Arrays: Uneven Number Of Elements

If the array has less elements than required, it will adjust from the end accordingly.

```
import numpy as np  
  
arr = np.array([1, 2, 3, 4, 5, 6])  
  
newarr = np.array_split(arr, 4)  
  
print(newarr)
```

```
[array([1, 2]), array([3, 4]), array([5]), array([6])]
```

Splitting Arrays: Accessing Resulting Arrays

Can access result arrays from the result just like any array element:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6])

newarr = np.array_split(arr, 3)

print(newarr[0])
print(newarr[1])
print(newarr[2])
```

[1 2]

[3 4]

[5 6]

Splitting Arrays: More Options

hsplit: split a 2-D array into multiple 2-D arrays along rows

vsplit: split a 2-D array into multiple 2-D arrays along columns

dsplit: split a 3-D (or more-D) array into multiple arrays along depth

Searching Arrays With **where**

Can search for a specific value

Get back a tuple

Need to dissect it

`(array([3, 5, 6]),)`

`item: [3 5 6]`

3

5

6

```
1 import numpy as np
2
3 arr = np.array([1, 2, 3, 4, 5, 4, 4])
4
5 x = np.where(arr == 4)
6
7 print(x)
8 print("")
9
10 for item in x:
11     print("item:", item, "\n|")
12     for index in item:
13         print(index)
14
```

Searching For Elements That Meet Some Criteria

Example: search for locations/indices of even numbers

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])
```

```
x = np.where(arr%2 == 0)
```

```
print(x)
```

```
(array([1, 3, 5, 7]),)
```

Sorting Arrays

Returns a COPY of the array, leaving original untouched

```
1 import numpy as np
2
3 arr = np.array([3, 2, 0, 1])
4
5 sorted_arr = np.sort(arr)
6
7 print("original array")
8 print(arr)
9 print("sorted array")
10 print(sorted_arr)
11 |
```

```
original array
[3 2 0 1]
sorted array
[0 1 2 3]
```

Sorting Arrays

Can sort strings or other types of data

```
import numpy as np
```

```
arr = np.array(['banana', 'cherry', 'apple'])
```

```
print(np.sort(arr))
```

```
['apple' 'banana' 'cherry']
```

```
import numpy as np
```

```
arr = np.array([True, False, True])
```

```
print(np.sort(arr))
```

```
[False True True]
```

Filtering Arrays

Creating a new array with some elements of an existing array

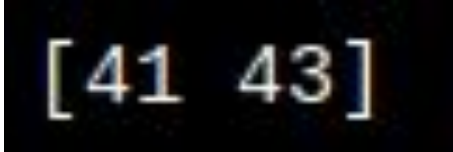
Use a list of boolean values to indicate what is included (True) and excluded (False) from the new array

```
import numpy as np

arr = np.array([41, 42, 43, 44])

x = arr[[True, False, True, False]]

print(x)
```



[41 43]

Filtering Arrays

Can create your own array of
booleans instead of
hard-coding it

```
import numpy as np

arr = np.array([41, 42, 43, 44])

# Create an empty list
filter_arr = []

# go through each element in arr
for element in arr:
    # if the element is higher than 42, set the value to True,
    # otherwise False:
    if element > 42:
        filter_arr.append(True)
    else:
        filter_arr.append(False)

newarr = arr[filter_arr]

print(filter_arr)
print(newarr)
```

```
[False, False, True, True]
[43 44]
```

Filtering Arrays: Alternate Method To Create Boolean Array

```
import numpy as np

arr = np.array([41, 42, 43, 44])

# Create an empty list
filter_arr = []

# go through each element in arr
for element in arr:
    # if the element is higher than 42, set the
    # otherwise False:
    if element > 42:
        filter_arr.append(True)
    else:
        filter_arr.append(False)

newarr = arr[filter_arr]

print(filter_arr)
print(newarr)
```

Create a filter array that will return only values higher than 42:

```
import numpy as np

arr = np.array([41, 42, 43, 44])

filter_arr = arr > 42

newarr = arr[filter_arr]

print(filter_arr)
print(newarr)
```

[False False True True]
[43 44]

Filtering Arrays: Alternate Method To Create Boolean Array

Create a filter array that will return only even elements from the original array:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7])

filter_arr = arr % 2 == 0

newarr = arr[filter_arr]

print(filter_arr)
print(newarr)
```

```
[False  True False  True False  True False]
[2  4  6]
```

NumPy Day 2 Lab Time!

Lab linked to Moodle

NumPy Day 3

Random Numbers

Pseudo random

- Generated by an algorithm
- Used for most applications

True random

- Needs input from outside source
- Used for security applications like encryption keys

NumPy gives pseudo random

Generating Random Ints & Floats

Generate a random integer from 0 to 100:

```
from numpy import random  
  
x = random.randint(100)  
  
print(x)
```

Generate a random float from 0 to 1:

```
from numpy import random  
  
x = random.rand()  
  
print(x)
```

Generating Arrays of Random Ints

Generate a 1-D array containing 5 random integers from 0 to 100:

```
from numpy import random  
  
x=random.randint(100, size=(5))  
  
print(x)
```

Generate a 2-D array with 3 rows, each row containing 5 random integers from 0 to 100:

```
from numpy import random  
  
x = random.randint(100, size=(3, 5))  
  
print(x)
```

Generating Arrays of Random Floats

Generate a 1-D array containing 5 random floats:

```
from numpy import random  
  
x = random.rand(5)  
  
print(x)
```

Generate a 2-D array with 3 rows, each row containing 5 random numbers:

```
from numpy import random  
  
x = random.rand(3, 5)  
  
print(x)
```

Pulling A Random Item Out Of An Array

The `choice()` method allows you to generate a random value based on an array of values.

The `choice()` method takes an array as a parameter and randomly returns one of the values.

Return one of the values in an array:

```
from numpy import random  
  
x = random.choice([3, 5, 7, 9])  
  
print(x)
```

Creating An Array Of Numbers Randomly From An Array

Generate a 2-D array that consists of the values in the array parameter (3, 5, 7, and 9):

```
from numpy import random  
  
x = random.choice([3, 5, 7, 9], size=(3, 5))  
  
print(x)
```

```
[[9 3 5 5 7]  
 [7 5 3 3 9]  
 [7 5 9 9 7]]
```

Random Permutations Of Elements

Permutations are orderings of elements

[1, 3, 2] is a permutation of [1, 2, 3]

[3, 2, 1] is a permutation of [1, 2, 3]

Etc.

Two methods in NumPy

Permutations & Shuffles

`shuffle()` changes an array

`permutation()` returns a new array (doesn't change the original array)

```
from numpy import random
import numpy as np

arr = np.array([1, 2, 3, 4, 5])

random.shuffle(arr)

print(arr)
```

```
from numpy import random
import numpy as np

arr = np.array([1, 2, 3, 4, 5])

print(random.permutation(arr))
```

Distributions (Ignoring The Math & Stats)

Distributions in MAT 130 and/or DSC 240

Saving explanation for those courses (better taught by Ph.D. mathematician)

You'll likely need to use some of the distributions they talk about at some point.

NumPy has your back!

Each distribution has parameter(s)

If you know the parameters, NumPy lets you create numbers from the distribution

Distributions

- Normal
- Binomial
- Poisson
- Uniform
- Logistic
- Multinomial
- Exponential
- Chi Square
- Rayleigh
- Pareto
- Zipf

Distributions

Need to make your own distribution? See this link:

https://www.w3schools.com/python/numpy/numpy_random_distribution.asp

Why Am I Not Showing You How?

You don't need to know now!

It's not hard.

They all look roughly like this (note only 1 real line of code)

Generate a random normal distribution of size 2x3 with mean at 1 and standard deviation of 2:

```
from numpy import random  
  
x = random.normal(loc=1, scale=2, size=(2, 3))  
  
print(x)
```

ufuncs (Universal Functions)

Faster way to manipulate arrays

Ex: add two arrays together

Uses the **add** ufunc

Other ways to do it (iterating over the arrays) are slower

```
import numpy as np
```

```
x = [1, 2, 3, 4]
```

```
y = [4, 5, 6, 7]
```

```
z = np.add(x, y)
```

```
print(z)
```

```
[ 5  7  9 11]
```

Arithmetic ufuncs

`add()`: sums the content of two arrays and returns the results in a new array

`subtract()`

`multiply()`

`divide()`

`power()`

`mod()`

`remainder()`

`divmod()`

`absolute()`

```
import numpy as np
```

```
arr1 = np.array([10, 11, 12, 13, 14, 15])
```

```
arr2 = np.array([20, 21, 22, 23, 24, 25])
```

```
newarr = np.add(arr1, arr2)
```

```
print(newarr)
```

```
[30 32 34 36 38 40]
```

Know These ufuncs Exist

Summing array elements

Multiplying array elements

Subtracting successive array elements

Logarithms

Least common multiple

Greatest common denominator

Rounding numbers

Trigonometric & Hyperbolic

Set operations (intersection, union, set difference, etc.)

NumPy Day 3 Lab Time!

Lab linked to Moodle

Pandas Day 1

What Is Pandas?

Pandas: a Python data analysis library

Good for dealing with structured data (data stored in tables, csv files, Excel sheets, database tables)

<https://pandas.pydata.org/>

- User guide
- Install
- Getting started
- etc.



<https://www.abc27.com/wp-content/uploads/sites/55/2023/03/GettyImages-1245864758.jpg?w=876&h=493&crop=1>

Remember To Import Pandas

```
import pandas as pd
```

Data Structures

Series: a one-dimensional labeled array holding data of any type such as integers, strings, Python objects etc.

DataFrame: a two-dimensional data structure that holds data like a two-dimensional array or a table with rows and columns.

Creating a Series

Create a simple Pandas Series from a list:

```
import pandas as pd

a = [1, 7, 2]

myvar = pd.Series(a)

print(myvar)
```

```
0    1
1    7
2    2
dtype: int64
```

Series Labels

Values in a series are labeled

- by default with index number
- or with labels you specify

Labels can be used to access values in series

```
import pandas as pd  
a = [1, 7, 2]  
myvar = pd.Series(a)  
print(myvar[0])
```



1

Series Labels

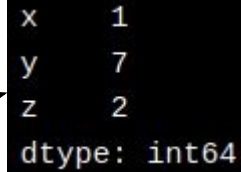
Make your own labels with the index argument

```
import pandas as pd

a = [1, 7, 2]

myvar = pd.Series(a, index = ["x", "y", "z"])

print(myvar)
```



x	1
y	7
z	2

dtype: int64

Access a value by referring to its label

```
print(myvar["y"])
```



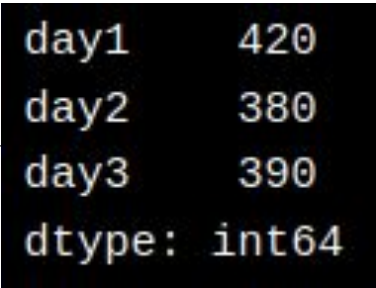
7

Key/Value Objects as Series

Can create a series from dictionary

Keys of dictionary become the labels

```
import pandas as pd  
calories = {"day1": 420, "day2": 380, "day3": 390}  
myvar = pd.Series(calories)  
print(myvar)
```



day1	420
day2	380
day3	390
dtype: int64	

Creating A Series From A Subset of A Dictionary

Use index argument to specify a subset of the dictionary

```
import pandas as pd  
  
calories = {"day1": 420, "day2": 380, "day3": 390}  
  
myvar = pd.Series(calories, index = ["day1", "day2"])  
  
print(myvar)
```

```
day1    420  
day2    380  
dtype: int64
```

DataFrames

Datasets in Pandas usually multi-dimensional tables called DataFrames

Series: like a column

DataFrame: whole table

Creating a DataFrame

Multiple ways to create a DF

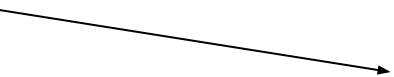
Can create from multiple Series

```
import pandas as pd

data = {
    "calories": [420, 380, 390],
    "duration": [50, 40, 45]
}

myvar = pd.DataFrame(data)

print(myvar)
```



	calories	duration
0	420	50
1	380	40
2	390	45

Manipulating DataFrames

Return a row as a Series

```
import pandas as pd

data = {
    "calories": [420, 380, 390],
    "duration": [50, 40, 45]
}

#load data into a DataFrame object:
df = pd.DataFrame(data)

print(df.loc[0])
```

calories	420
duration	50
Name: 0, dtype: int64	

Return a DataFrame

```
print(df.loc[[0,1]])
```

	calories	duration
0	420	50
1	380	40

Note the list of indices...

Naming Indexes For A DataFrame

```
import pandas as pd

data = {
    "calories": [420, 380, 390],
    "duration": [50, 40, 45]
}

myvar = pd.DataFrame(data)

print(myvar)
```

	calories	duration
0	420	50
1	380	40
2	390	45

```
import pandas as pd

data = {
    "calories": [420, 380, 390],
    "duration": [50, 40, 45]
}

df = pd.DataFrame(data, index = ["day1", "day2", "day3"])

print(df)
```

	calories	duration
day1	420	50
day2	380	40
day3	390	45

Locating Named Indexes

```
import pandas as pd

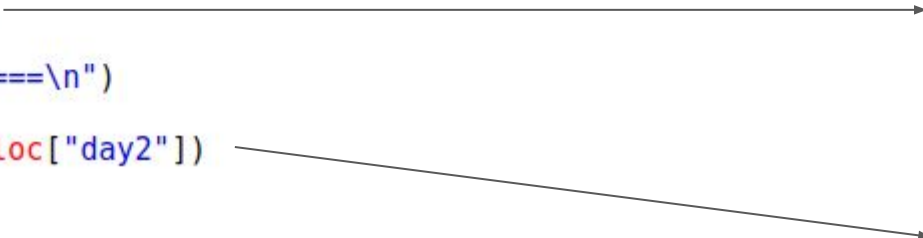
data = {
    "calories": [420, 380, 390],
    "duration": [50, 40, 45]
}

df = pd.DataFrame(data, index = ["day1", "day2", "day3"])

print(df)

print("\n===\n")

print(df.loc["day2"])
```



	calories	duration
day1	420	50
day2	380	40
day3	390	45

===

calories	380
duration	40
Name: day2, dtype: int64	

Loading A CSV File Into A DataFrame

```
import pandas as pd
```

```
df = pd.read_csv('data.csv')
```

```
print(df)
```

data.csv

Duration,Pulse,Maxpulse,Calories

60,110,130,409.1
60,117,145,479.0
60,103,135,340.0
45,109,175,282.4
45,117,148,406.0
60,102,127,300.5
60,110,136,374.0
45,104,134,253.3
30,109,133,195.1
60,98,124,269.0
60,103,147,329.3
60,100,120,250.7
60,106,128,345.3
60,104,132,379.3
60,98,123,275.0
60,98,120,215.2
60,100,120,300.0
45,90,112,
60,103,123,333.0

	Duration	Pulse	Maxpulse	Calories
0	60	110	130	409.1
1	60	117	145	479.0
2	60	103	135	340.0
3	45	109	175	282.4
4	45	117	148	406.0
..	...	...	...	...
164	60	105	140	290.8
165	60	110	145	300.4
166	60	115	145	310.2
167	75	120	150	320.4
168	75	125	150	330.4

[169 rows x 4 columns]

Limitations Of Printing Your DataFrame

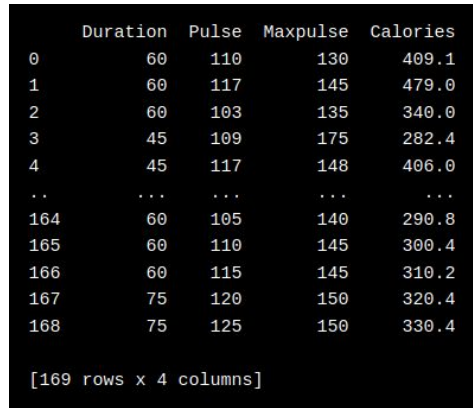
If DF has too many rows, Pandas only prints header, first 5, last 5 rows

To print whole DF, use `to_string()`

```
import pandas as pd

df = pd.read_csv('data.csv')

print(df.to_string())
```



	Duration	Pulse	Maxpulse	Calories
0	60	110	130	409.1
1	60	117	145	479.0
2	60	103	135	340.0
3	45	109	175	282.4
4	45	117	148	406.0
..	...	...	...	...
164	60	105	140	290.8
165	60	110	145	300.4
166	60	115	145	310.2
167	75	120	150	320.4
168	75	125	150	330.4

[169 rows x 4 columns]

Can find & adjust what “too many” means

Finding & Adjusting “Too Many”

Finding

```
import pandas as pd  
  
print(pd.options.display.max_rows)
```

Adjusting

```
import pandas as pd  
  
pd.options.display.max_rows = 9999  
  
df = pd.read_csv('data.csv')  
  
print(df)
```

Head & Tail

Want to print the first n rows or last n rows?

```
import pandas as pd

df = pd.read_csv('data.csv')

print(df.head(12))

print("\n====\n")

print(df.tail(7))
```

If you don't specify a value inside parens when using head or tail, default is 5.

	Duration	Pulse	Maxpulse	Calories
0	60	110	130	409.1
1	60	117	145	479.0
2	60	103	135	340.0
3	45	109	175	282.4
4	45	117	148	406.0
5	60	102	127	300.5
6	60	110	136	374.0
7	45	104	134	253.3
8	30	109	133	195.1
9	60	98	124	269.0
10	60	103	147	329.3
11	60	100	120	250.7
====				
	Duration	Pulse	Maxpulse	Calories
162	45	95	130	270.0
163	45	100	140	280.9
164	60	105	140	290.8
165	60	110	145	300.4
166	60	115	145	310.2
167	75	120	150	320.4
168	75	125	150	330.4

Creating DF's From JSON Files

JSON file is common storage for big data

Beyond scope for now (maybe during advanced topics)

If you need it later: https://www.w3schools.com/python/pandas/pandas_json.asp

Pandas Day 1 Lab Time!

Lab linked to Moodle

Pandas Day 2

Quick Review Of Some Topics

1. Get a DataFrame from a CSV file.
2. Get a row as a Series
3. Get a DataFrame subset of a DataFrame

```
toth@toth-Latitude-5520 ~  
$ python3 pandaslec.py  
   Player  Goals  Assists  Yellow Cards  Red Cards  Position  Team  
0  Millie Bright    0      0           5           1         DF   Chelsea  
1  Erin Cuthbert    3      4           2           0         MF   Chelsea  
2  Samantha Kerr   10      0           2           0         FW   Chelsea  
3  Katie McCabe    3      1           5           1         DF   Arsenal  
4  Chloe Kelly     5      3           3           1         FW  Manchester City  
  
===  
Player      Erin Cuthbert  
Goals              3  
Assists            4  
Yellow Cards       2  
Red Cards          0  
Position           MF  
Team              Chelsea  
Name: 1, dtype: object  
  
===  
   Player  Goals  Assists  Yellow Cards  Red Cards  Position  Team  
1  Erin Cuthbert    3      4           2           0         MF   Chelsea  
3  Katie McCabe    3      1           5           1         DF   Arsenal
```

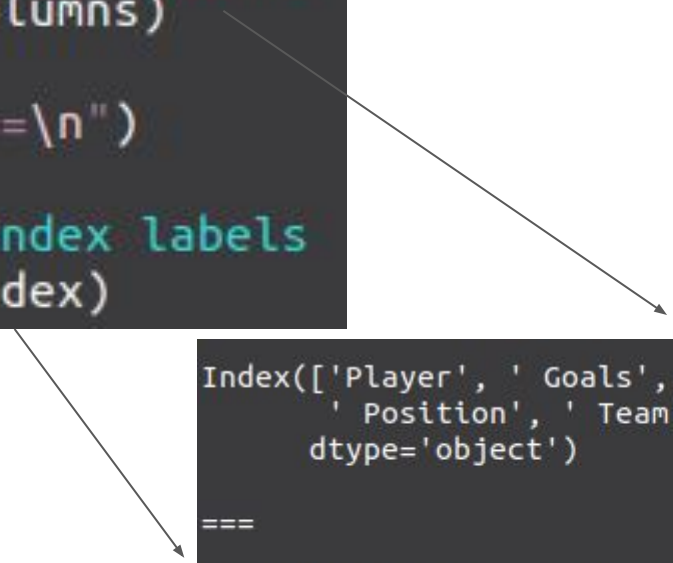
```
import pandas as pd  
  
# Read a CSV file into a DataFrame  
df = pd.read_csv('soccer.csv')  
  
print(df)  
  
print("\n===\n")  
  
# Get a row as a Series  
some_player = df.loc[1]  
  
print(some_player)  
  
print("\n===\n")  
  
# Get a DataFrame  
some_subset = df.loc[[1,3]]  
  
print(some_subset)
```

Getting Info About Your DataFrame

```
# Display column names
print(df.columns)

print("\n===\n")

# Display index labels
print(df.index)
```



```
Index(['Player', 'Goals', 'Assists', 'Yellow Cards', 'Red Cards',
       'Position', 'Team'],
      dtype='object')

===

RangeIndex(start=0, stop=5, step=1)
```

Getting Statistical Summary Info About Your DataFrame

```
# Print statistical summary of the DataFrame  
print(df.describe())
```

	Goals	Assists	Yellow Cards	Red Cards
count	5.000000	5.000000	5.000000	5.000000
mean	4.200000	1.600000	3.400000	0.600000
std	3.701351	1.81659	1.516575	0.547723
min	0.000000	0.000000	2.000000	0.000000
25%	3.000000	0.000000	2.000000	0.000000
50%	3.000000	1.000000	3.000000	1.000000
75%	5.000000	3.000000	5.000000	1.000000
max	10.000000	4.000000	5.000000	1.000000

Getting Information About Your Data Set

`print(df.info())` will give you

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 169 entries, 0 to 168  
Data columns (total 4 columns):  
#   Column      Non-Null Count  Dtype  
---  -  
0   Duration    169 non-null    int64  
1   Pulse       169 non-null    int64  
2   Maxpulse    169 non-null    int64  
3   Calories    164 non-null    float64  
dtypes: float64(1), int64(3)  
memory usage: 5.4 KB  
None
```

169 rows
4 columns

name & data type of each column

How many non-null values per column
(thus 5 rows in Calories column have missing values)

Indexing And Selecting Data

.loc: label-based

- Single label like 5 or 'some_text' (for 5, it's assumes 5 is the label of the index, not the position along the index)
- List or array of labels
- Slice of labels 'a':'b' - note unlike usual slice, both a and b ARE included!

.iloc: integer position-based

- An int like 5
- A list or array of ints
- A slice with ints like 1:7

.loc Indexing And Selecting Data Examples

.loc: label-based

- Single label like 5 or 'some_text' (for 5, it's assumes 5 is the label of the index, not the position along the index)
- List or array of labels
- Slice of labels 'a':'b' - note unlike usual slice, both a and b ARE included!

Could also do a pair of slices in here like ['a':'b', 'B':'C'] instead of ['a', 'B'] or could also do a list of lists in here like [['a', 'b', 'c'], ['B', 'D']] instead of ['a', 'B']

```
import pandas as pd
import numpy as np

df1 = pd.DataFrame(np.random.randint(10, 99, (6, 4)),\
                    index=list('abcdef'), columns=list('ABCD'))

print(df1)

print ("\n")

print(df1.loc['a'])

print("\n")

print(df1.loc['a', 'B'])

print("\n")

print(df1.loc['b':'e'])
```

toth@toth-MINI-S:~\$ python3 tesloc.py

	A	B	C	D
a	29	86	98	87
b	19	89	72	43
c	17	11	26	81
d	64	38	62	56
e	66	86	97	12
f	11	89	85	82

A 29
B 86
C 98
D 87
Name: a, dtype: int64

86

	A	B	C	D
b	19	89	72	43
c	17	11	26	81
d	64	38	62	56
e	66	86	97	12

toth@toth-MINI-S:~\$

.iloc Indexing And Selecting Data Examples

.iloc: integer position-based

- An int like 5
- A list or array of ints
- A slice with ints like 1:7

Could also do a pair of slices in here like `[x:y, z:a]` instead of `[1:3]`
or could also do a list of lists in here like `[[1, 3, 4], [1, 2]]` instead of `[1:3]`

```
import pandas as pd
import numpy as np

df1 = pd.DataFrame(np.random.randint(10, 99, (6, 4)),\
                    index=list('abcdef'), columns=list('ABCD'))

print(df1)

print("\n")

print(df1.iloc[3])

print("\n")

print(df1.iloc[1, 3])

print("\n")

print(df1.iloc[2:4])
```

toth@toth-MINI-S:~\$ python3 tesloc.py

	A	B	C	D
a	41	77	30	22
b	49	24	74	82
c	77	15	65	46
d	19	69	86	11
e	36	83	20	46
f	81	27	41	92

A 19
B 69
C 86
D 11
Name: d, dtype: int64

82

	A	B	C	D
c	77	15	65	46
d	19	69	86	11

toth@toth-MINI-S:~\$

Set / Reset Index

https://pandas.pydata.org/docs/user_guide/indexing.html#set-reset-index

As a lab activity, you will explore this in Question 10

The Easiest Way To Screw Up...

“The inmates are running the asylum”

Programmers are generally not UI/UX people

They do things that make programming sense but are challenging for the user

Inconsistent interfaces & APIs are terrible

Pandas is inconsistent from a user perspective

Some functions return a new DataFrame but others modify an existing one!

Sorting Your DataFrame With sort_values()

sort_values() does not modify your DataFrame - it returns a new one!

```
# Sort the DataFrame by values
my_new_df = df.sort_values(by=" Assists")
print(my_new_df)
```

Be really careful with column names!
Note the blank space before Assists!



	Player	Goals	Assists	Yellow Cards	Red Cards	Position	Team
0	Millie Bright	0	0	5	1	DF	Chelsea
2	Samantha Kerr	10	0	2	0	FW	Chelsea
3	Katie McCabe	3	1	5	1	DF	Arsenal
4	Chloe Kelly	5	3	3	1	FW	Manchester City
1	Erin Cuthbert	3	4	2	0	MF	Chelsea

What happens if we just do df.sort_values(by=" Assists")?

Can I Sort By Multiple Columns?

Great question! Remember, we're seeing a subset of Pandas. How can we find out? RTFM or STFW!

```
# Sort the DataFrame by values
my_new_df = df.sort_values(by=" Assists")

print(my_new_df)

print("\n===\n")

# Sort the DataFrame by values
my_new_df2 = df.sort_values(by=[" Assists", " Yellow Cards"])

print(my_new_df2)
```

	Player	Goals	Assists	Yellow Cards	Red Cards	Position	Team
2	Millie Bright	0	0	5	1	DF	Chelsea
2	Samantha Kerr	10	0	2	0	FW	Chelsea
3	Katie McCabe	3	1	5	1	DF	Arsenal
4	Chloe Kelly	5	3	3	1	FW	Manchester City
1	Erin Cuthbert	3	4	2	0	MF	Chelsea

===

	Player	Goals	Assists	Yellow Cards	Red Cards	Position	Team
2	Samantha Kerr	10	0	2	0	FW	Chelsea
0	Millie Bright	0	0	5	1	DF	Chelsea
3	Katie McCabe	3	1	5	1	DF	Arsenal
4	Chloe Kelly	5	3	3	1	FW	Manchester City
1	Erin Cuthbert	3	4	2	0	MF	Chelsea

So can we sort from high to low instead of low to high?

We'll see in lab!

Getting A Scalar With at And iat

at: label-based lookup (similar to loc)

```
some_value = some_df.at['a', 'V']
```

iat: position-based lookup (similar to iloc)

```
some_value = some_df.iat[4, 5]
```

Getting A Scalar With at And iat

How many assists does Chloe Kelly have?

	Player	Goals	Assists	Yellow Cards	Red Cards	Position	Team
0	Millie Bright	0	0	5	1	DF	Chelsea
2	Samantha Kerr	10	0	2	0	FW	Chelsea
3	Katie McCabe	3	1	5	1	DF	Arsenal
4	Chloe Kelly	5	3	3	1	FW	Manchester City
1	Erin Cuthbert	3	4	2	0	MF	Chelsea

She's at row 4, col 3

```
cka = df.iat[4, 2]
print("Chloe Kelly's assists:", cka)
```

Chloe Kelly's assists: 3

Cleaning Data

Dealing With Empty Cells: Detection

- `isna()`
- `notna()`
- `notnull()`

See https://www.w3schools.com/python/pandas/pandas_ref_dataframe.asp for those methods and lots of other ones for DataFrames

Dealing With Empty Cells

Removing a small number of rows if there are tons is usually ok

Return a new Data Frame with no empty cells:

```
import pandas as pd

df = pd.read_csv('data.csv')

new_df = df.dropna()

print(new_df.to_string())
```

dropna() returns a new DataFrame - doesn't change old one, unless

Remove all rows with NULL values:

```
import pandas as pd

df = pd.read_csv('data.csv')

df.dropna(inplace = True)

print(df.to_string())
```

before

after

Duration,Date,Pulse,Maxpulse,Calories

```
60, '2020/12/01', 110, 130, 409.1
60, '2020/12/02', 117, 145, 479.0
60, '2020/12/03', 103, 135, 340.0
45, '2020/12/04', 109, 175, 282.4
45, '2020/12/05', 117, 148, 406.0
60, '2020/12/06', 102, 127, 300.0
60, '2020/12/07', 110, 136, 374.0
450, '2020/12/08', 104, 134, 253.3
30, '2020/12/09', 109, 133, 195.1
60, '2020/12/10', 98, 124, 269.0
60, '2020/12/11', 103, 147, 329.3
60, '2020/12/12', 100, 120, 250.7
60, '2020/12/12', 100, 120, 250.7
60, '2020/12/13', 106, 128, 345.3
60, '2020/12/14', 104, 132, 379.3
60, '2020/12/15', 98, 123, 275.0
60, '2020/12/16', 98, 120, 215.2
60, '2020/12/17', 100, 120, 300.0
45, '2020/12/18', 90, 112,
60, '2020/12/19', 103, 123, 323.0
45, '2020/12/20', 97, 125, 243.0
60, '2020/12/21', 108, 131, 364.2
45, 100, 119, 282.0
60, '2020/12/23', 130, 101, 300.0
45, '2020/12/24', 105, 132, 246.0
60, '2020/12/25', 102, 126, 334.5
60, 20201226, 100, 120, 250.0
60, '2020/12/27', 92, 118, 241.0
60, '2020/12/28', 103, 132,
60, '2020/12/29', 100, 132, 280.0
60, '2020/12/30', 102, 129, 380.3
60, '2020/12/31', 92, 115, 243.0
```

Dealing With Empty Cells

	Duration	Date	Pulse	Maxpulse	Calories
0	60	'2020/12/01'	110	130	409.1
1	60	'2020/12/02'	117	145	479.0
2	60	'2020/12/03'	103	135	340.0
3	45	'2020/12/04'	109	175	282.4
4	45	'2020/12/05'	117	148	406.0
5	60	'2020/12/06'	102	127	300.0
6	60	'2020/12/07'	110	136	374.0
7	450	'2020/12/08'	104	134	253.3
8	30	'2020/12/09'	109	133	195.1
9	60	'2020/12/10'	98	124	269.0
10	60	'2020/12/11'	103	147	329.3
11	60	'2020/12/12'	100	120	250.7
12	60	'2020/12/12'	100	120	250.7
13	60	'2020/12/13'	106	128	345.3
14	60	'2020/12/14'	104	132	379.3
15	60	'2020/12/15'	98	123	275.0
16	60	'2020/12/16'	98	120	215.2
17	60	'2020/12/17'	100	120	300.0
18	60	'2020/12/18'	90	112	
19	60	'2020/12/19'	103	123	323.0
20	45	'2020/12/20'	97	125	243.0
21	60	'2020/12/21'	108	131	364.2
22	60	'2020/12/22'	100	120	300.0
23	60	'2020/12/23'	130	101	300.0
24	45	'2020/12/24'	105	132	246.0
25	60	'2020/12/25'	102	126	334.5
26	60	'2020/12/26'	100	120	250.0
27	60	'2020/12/27'	92	118	241.0
28	60	'2020/12/28'	103	132	
29	60	'2020/12/29'	100	132	280.0
30	60	'2020/12/30'	102	129	380.3
31	60	'2020/12/31'	92	115	243.0

Dealing With Empty Cells

Can replace the missing value instead of deleting the row - consult a statistician before doing this!!!

Replace NULL values with the number 130:

```
import pandas as pd

df = pd.read_csv('data.csv')

df.fillna(130, inplace = True)
```

Replace NULL values in the "Calories" columns with the number 130:

```
import pandas as pd

df = pd.read_csv('data.csv')

df["Calories"].fillna(130, inplace = True)
```

Dealing With Empty Cells

Can replace the missing value instead of deleting the row - consult a statistician before doing this!!!

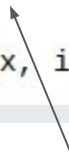
Calculate the MEAN, and replace any empty values with it:

```
import pandas as pd

df = pd.read_csv('data.csv')

x = df["Calories"].mean()

df["Calories"].fillna(x, inplace = True)
```



Or median or mode - again, consult statistician!

Dealing With Incorrectly Formatted Data

Remove it

Remove rows with a NULL value in the "Date" column:

```
df.dropna(subset=['Date'], inplace = True)
```

21	60	'2020/12/21'	108	131	364.2
22	45	NaN	100	119	282.0
23	60	'2020/12/23'	130	101	300.0
24	45	'2020/12/24'	105	132	246.0
25	60	'2020/12/25'	102	126	334.5
26	60	20201226	100	120	250.0

21	60	2020-12-21	108	131	364.2
23	60	2020-12-23	130	101	300.0
24	45	2020-12-24	105	132	246.0

22 is gone!

or convert it!

```
df['Date'] = pd.to_datetime(df['Date'])
```

21	60	2020-12-21	108	131	364.2
22	45	NaT	100	119	282.0
23	60	2020-12-23	130	101	300.0
24	45	2020-12-24	105	132	246.0
25	60	2020-12-25	102	126	334.5
26	60	2020-12-26	100	120	250.0

Dealing With Wrong Data

If you know what's wrong & how to fix it

Set "Duration" = 45 in row 7:

```
df.loc[7, 'Duration'] = 45
```

or

Loop through all values in the "Duration" column.

If the value is higher than 120, set it to 120:

```
for x in df.index:
    if df.loc[x, "Duration"] > 120:
        df.loc[x, "Duration"] = 120
```

	Duration	Date	Pulse	Maxpulse	Calories
0	60	'2020/12/01'	110	130	409.1
1	60	'2020/12/02'	117	145	479.0
2	60	'2020/12/03'	103	135	340.0
3	45	'2020/12/04'	109	175	282.4
4	45	'2020/12/05'	117	148	406.0
5	60	'2020/12/06'	102	127	300.0
6	60	'2020/12/07'	110	136	374.0
7	450	'2020/12/08'	104	134	253.3
8	30	NaN	109	133	195.1
9	60	'2020/12/10'	98	124	269.0
10	60	'2020/12/11'	103	147	329.3
11	9999	'2020/12/12'	100	120	250.7
12	9999	'2020/12/12'	100	120	250.7
13	60	'2020/12/13'	106	128	345.3

	Duration	Date	Pulse	Maxpulse	Calories
0	60	'2020/12/01'	110	130	409.1
1	60	'2020/12/02'	117	145	479.0
2	60	'2020/12/03'	103	135	340.0
3	45	'2020/12/04'	109	175	282.4
4	45	'2020/12/05'	117	148	406.0
5	60	'2020/12/06'	102	127	300.0
6	60	'2020/12/07'	110	136	374.0
7	120	'2020/12/08'	104	134	253.3
8	30	'2020/12/09'	109	133	195.1
9	60	'2020/12/10'	98	124	269.0

Detecting & Removing Duplicates

Detect (get back a series): or

```
my_dups = df.duplicated()
```

```
print(my_dups)
```

Remove

```
df.drop_duplicates(inplace = True)
```

Note the inplace!!!

Modifies df!

```
import pandas as pd

df = pd.read_csv('data.csv')

print(df.to_string())

print("\n===\n")

print(df.duplicated())

print("\n===\n")

df.drop_duplicates(inplace = True)

print(df.to_string())
```

12 is missing! →

```
tothetoth-MINI-S:~$ python3 dups.py
Duration      Date      Pulse  Maxpulse  Calories
0          60  '2020/12/01'    110      130     409.1
1          60  '2020/12/02'    117      145     479.0
2          60  '2020/12/03'    103      135     340.0
3          45  '2020/12/04'    109      175     282.4
4          45  '2020/12/05'    117      148     406.0
5          60  '2020/12/06'    102      127     300.0
6          60  '2020/12/07'    110      136     374.0
7         450  '2020/12/08'    104      134     253.3
8          30             NaN    109      133     195.1
9          60  '2020/12/10'     98      124     269.0
10         60  '2020/12/11'    103      147     329.3
11       9999  '2020/12/12'    100      120     250.7
12       9999  '2020/12/12'    100      120     250.7
13          60  '2020/12/13'    106      128     345.3

===
0      False
1      False
2      False
3      False
4      False
5      False
6      False
7      False
8      False
9      False
10     False
11     False
12      True
13     False
dtype: bool

===
Duration      Date      Pulse  Maxpulse  Calories
0          60  '2020/12/01'    110      130     409.1
1          60  '2020/12/02'    117      145     479.0
2          60  '2020/12/03'    103      135     340.0
3          45  '2020/12/04'    109      175     282.4
4          45  '2020/12/05'    117      148     406.0
5          60  '2020/12/06'    102      127     300.0
6          60  '2020/12/07'    110      136     374.0
7         450  '2020/12/08'    104      134     253.3
8          30             NaN    109      133     195.1
9          60  '2020/12/10'     98      124     269.0
10         60  '2020/12/11'    103      147     329.3
11       9999  '2020/12/12'    100      120     250.7
13          60  '2020/12/13'    106      128     345.3

tothetoth-MINI-S:~$
```

Detecting & Removing Duplicates

Syntax: `DataFrame.drop_duplicates(subset=None, keep='first', inplace=False)`

Parameters:

- **subset:** Subset takes a column or list of column label. It's default value is none. After passing columns, it will consider them only for duplicates.
- **keep:** keep is to control how to consider duplicate value. It has only three distinct value and default is 'first'.
 - If **'first'**, it considers first value as unique and rest of the same values as duplicate.
 - If **'last'**, it considers last value as unique and rest of the same values as duplicate.
 - If **False**, it consider all of the same values as duplicates
- **inplace:** Boolean values, removes rows with duplicates if True.

Return type: DataFrame with removed duplicate rows depending on Arguments passed.

Pandas Day 2 Lab Time!

Lab linked to Moodle

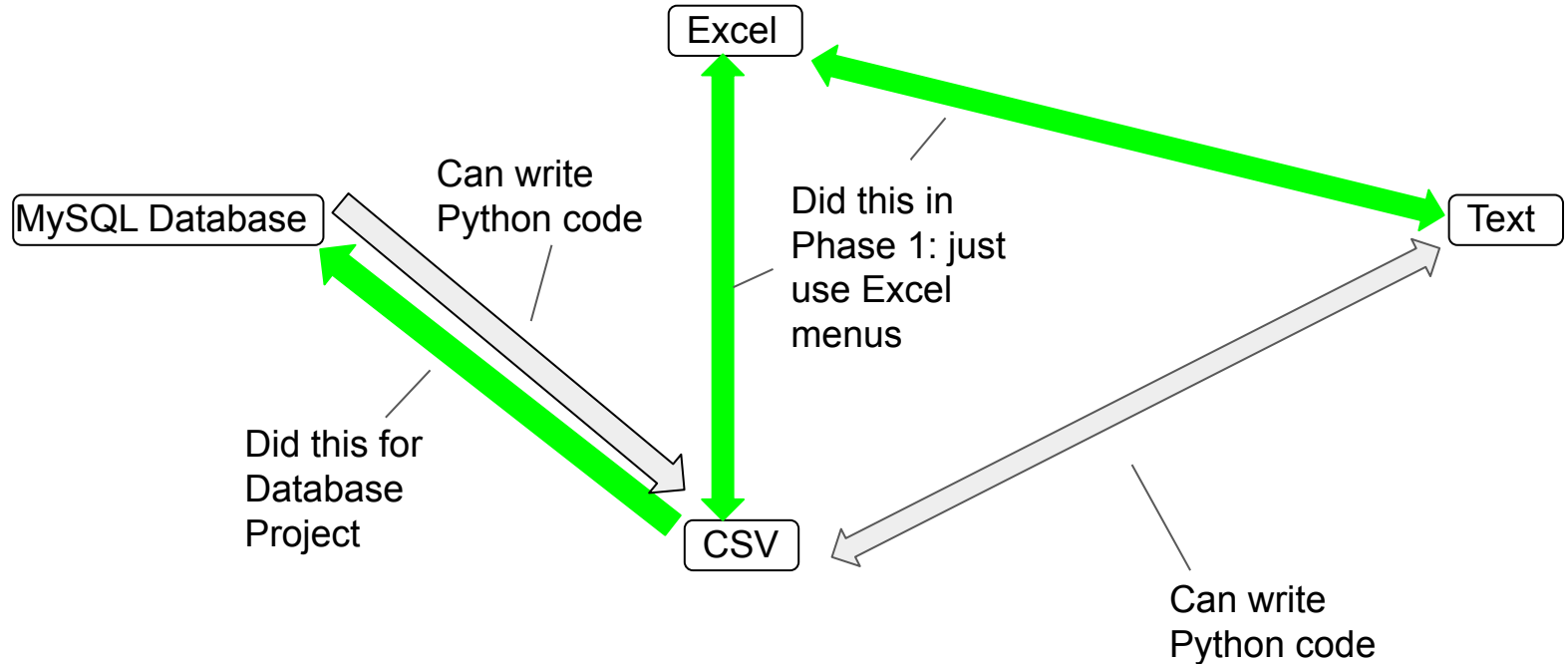
Pandas Day 3

Moving Data Between Data Sources

How do you get your data when you do a project?

- Text files
- CSV files
- Excel files
- A MySQL database
- HTML pages (later in advanced topics)
- JSON files (later in advanced topics)

Moving Data Between Data Sources



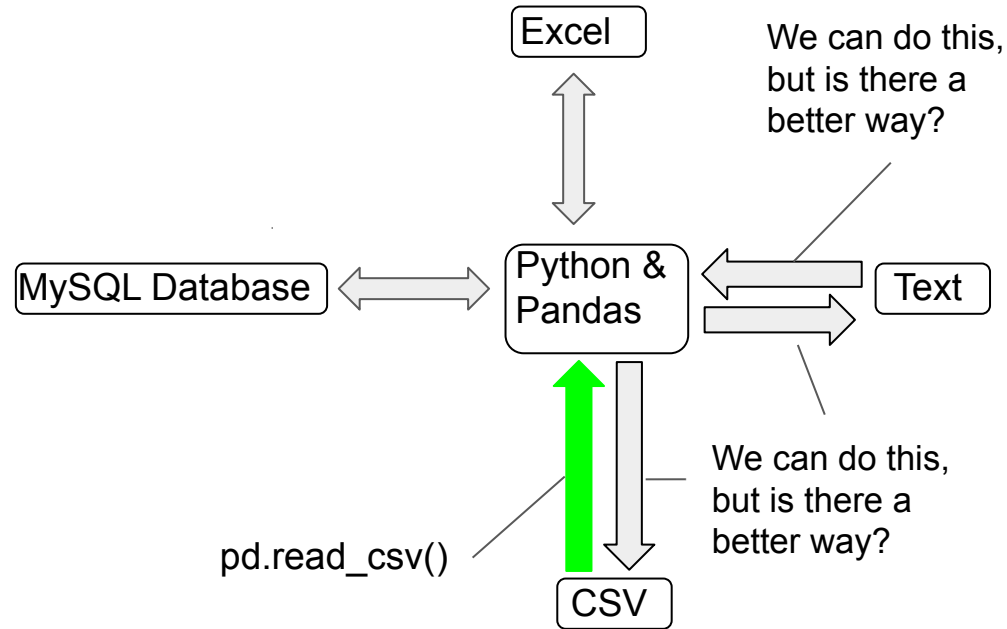
Moving Data Into Programs

How will you process your data?

- Excel (a lot less)
- Python & Pandas (a lot more)

What's the next step?

- Get data into & out of Python & Pandas



How We Want Our Data

When getting data, want it in DataFrame

When exporting data, want it in some predictable & useful format (Text, CSV, Excel, or Database)

CSV File ↔ Pandas

Import from CSV file (review):

```
df = pd.read_csv('soccer.csv')
```

Export to CSV file:

```
result_dataframe.to_csv('output.csv')
```

Just be careful about what the output file looks like (index gets saved, so if you reimport...)

Text File ↔ Pandas

Import from text:

```
newdf = pd.read_csv("soccer_tab.txt", sep="\t")
```

<https://www.geeksforgeeks.org/how-to-read-text-files-with-pandas/#>

Export to text:

```
df.to_csv('socoutput.txt', sep='\t')
```

<https://copyprogramming.com/howto/python-export-dataframe-to-txt-in-python>

Excel File Pandas

Import from Excel:

```
df = pd.read_excel("employees.xlsx", sheet_name = 0)
```

<https://stackoverflow.com/questions/72588629/convert-an-excel-file-to-a-pandas-data-frame-by-asking-user-to-choose-excel-work>

Export to Excel:

<https://blog.enterprisedna.co/how-to-export-a-pandas-dataframe-to-excel-in-python/>

MySQL Database ↔ Pandas: Get Data From Database

```
import MySQLdb      # swapping the order of these imports causes an error on  
import pandas as pd  # my computer
```

```
query = "select * from customers"
```

```
db = MySQLdb.connect("10.14.1.32", "toth", "DSC270F@112023", "classicmodels")
```

```
result_dataFrame = pd.read_sql(query,db)
```

```
db.close()
```

```
print(result_dataFrame)
```

server, database username, database password, database
(same as when we did it with Python but without Pandas)

MySQL Database ↔ Pandas: Write To Database

Sources:

<https://www.dataquest.io/blog/sql-insert-tutorial/>

<https://www.kaggle.com/code/dilarabr/writing-pandas-dataframe-to-mysql>

Special characters in passwords can cause issues:

<https://stackoverflow.com/questions/71398129/cant-connect-to-mysql-db-with-python-pandas-due-to-in-the-password>

<https://stackoverflow.com/questions/23406953/how-do-i-escape-letter-from-sql-password-in-connection-uri/23408743#23408743>

<https://stackoverflow.com/questions/31827012/python-importing-urllib-quote>

MySQL Database ↔ Pandas: Write To Database

Source

```
import MySQLdb
import pandas as pd
from sqlalchemy import create_engine
import urllib.parse

newdf = pd.read_csv("soccer_tab.txt", sep="\t")
print(newdf)

engine = create_engine("mysql+pymysql://{user}:{pw}@10.14.1.32/{db}"\
    .format(user="toth",\
        pw=urllib.parse.quote("DSC270F@ll2023"),db="soccer"))

newdf.to_sql('player', con = engine, if_exists='append', index=False)
```

My data file

Database
server

Database

Database table

Database username

Database password

Pandas Day 3 Lab Time!

Lab linked to Moodle

Pandas Day 4

Topics For Today

Continuing

1. Finishing Monday's lab

New Topics

2. Renaming columns
3. Counting values
4. Making a new column from existing columns
5. Adding new columns to an existing DataFrame
6. Deleting rows and columns from a DataFrame
7. Conditional slicing/selections
8. Concatenating DataFrames

Today's Methods

We're all going to be out of synch because of Monday's snafu.

I need to be free to help finish up the database portion from Monday.

We'll do a self-guided lesson so everybody can proceed at their own pace.

This is a also great way to nudge you in the direction of self-sufficiency!

Pandas Day 4 Lab Time!

1 Finish Monday's lab linked to Moodle

2 Do today's lab linked to Moodle

My suggestion is to walk through the lab question by question.

As you get to a question in the lab handout, look at the slide, chase down any link on it, do the reading from the link, and then try the lab question

Renaming Columns

<https://www.datacamp.com/tutorial/pandas>

There's a ton of info on this page you'll need to either read or skim through it (takes a long time) or do something clever: ***control-f*** will let you search the page for text...

Counting Values

This link

[https://www.datacamp.com/tutorial/pandas#counting-using-.value_counts\(\)](https://www.datacamp.com/tutorial/pandas#counting-using-.value_counts()) will take you directly to what you need: **Counting using `.value_counts()`**

You should read up to but NOT including **Aggregating data with `.groupby()` in pandas**

Making A New Column From Existing Columns

A really short, to-the-point set of instructions:

<https://www.datacamp.com/tutorial/pandas#create-new-columns-based-on-existing-columns>

Adding Columns To An Existing DataFrame

Note there are different methods shown here! You should look at them all!

<https://www.machinelearningplus.com/pandas/pandas-add-column/>

Deleting Rows And Columns From A DataFrame

<https://www.geeksforgeeks.org/python-delete-rows-columns-from-dataframe-using-pandas-drop/>

Conditional Slicing/Selections

[https://www.datacamp.com/tutorial/pandas#conditional-slicing-\(that-fits-certain-conditions\)](https://www.datacamp.com/tutorial/pandas#conditional-slicing-(that-fits-certain-conditions))

Another place where control-f will serve you well.

<https://www.learndatasci.com/tutorials/python-pandas-tutorial-complete-introduction-for-beginners/>

Concatenating DataFrames

<https://www.simplilearn.com/tutorials/python-tutorial/python-pandas>

control-f

String Manipulation

<https://www.geeksforgeeks.org/string-manipulations-in-pandas-dataframe/#>

Pandas Day 5

Reproducibility

```
rng = np.random.RandomState(42)  
ser = pd.Series(rng.rand(5))    OR  
ser = pd.Series(rng.randint(10, size=(5)))  
print(ser)
```

Always gets same random numbers!

```
df = pd.DataFrame({'A': rng.rand(5), 'B': rng.rand(5)})    OR  
df = pd.DataFrame({'A': rng.randint(10, size=(5)), 'B': rng.randint(10, size=(5))})  
print(df)
```

Aggregation

`df.mean()` will get mean of each column of DataFrame `df`

`df.mean(axis = 'columns')` will get mean of each row of DataFrame `df`

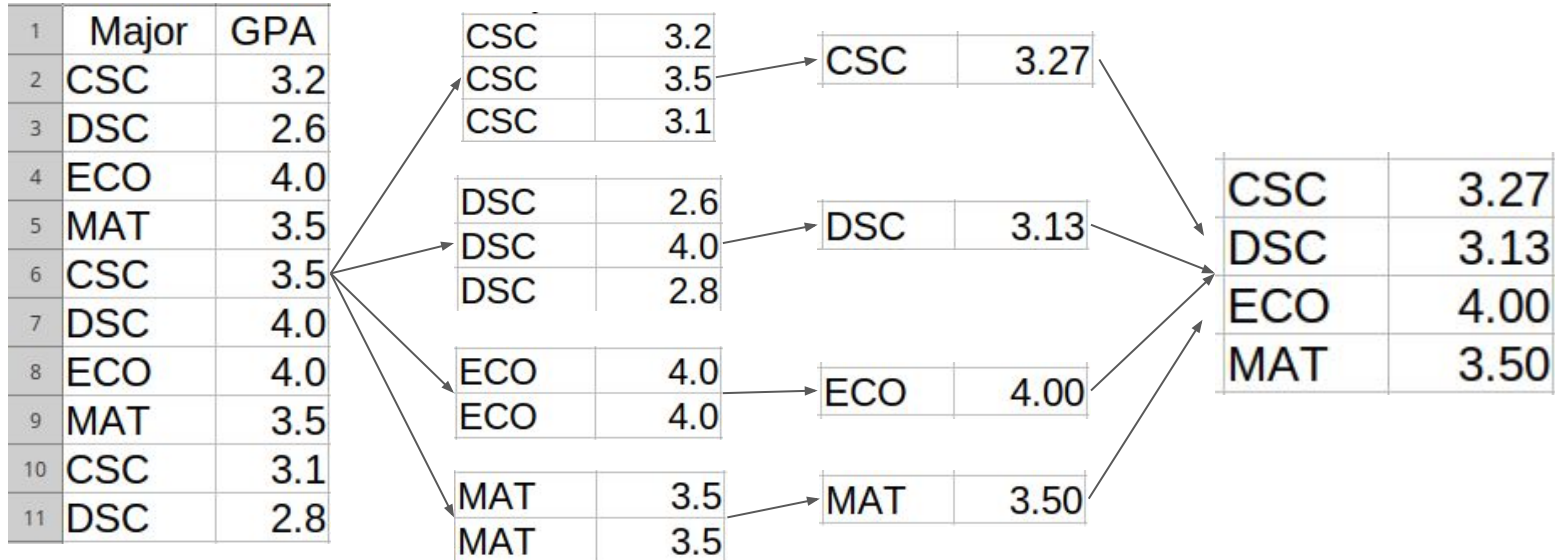
Also works with

- `count()`
- `first()`
- `std()`
- `prod()`
- `etc`

GroupBy

Might want avg GPA by major rather than avg for all majors

groupby will **split** our DF into each major, **apply** the avg function to each group, and **combine** the results into a new DF all with one command



GroupBy

```
df = pd.DataFrame({'Major': ['CSC', 'DSC', 'ECO', 'MAT', 'CSC', 'DSC', 'ECO', 'MAT', 'CSC', 'DSC'],\
                    'GPA': [3.2, 2.6, 4.0, 3.5, 3.5, 4.0, 4.0, 3.5, 3.1, 2.8]}, columns=['Major', 'GPA'])

print(df)

new_df = df.groupby('Major').mean()

print("\n===\n")

print(new_df)|
```

Remind you of GROUP BY from SQL?

```
mysql> SELECT team, MAX(goals) FROM players GROUP BY team;
+-----+-----+
| team      | MAX(goals) |
+-----+-----+
| Chelsea   | 4          |
| Aston Villa | 12         |
| Arsenal   | 5          |
+-----+-----+
3 rows in set (0.00 sec)

mysql> |
```

Yes, that's ugly. It's because Pandas creates an object, not a DataFrame. So pretending it's a DataFrame is not really the best idea. You can do anything to it you would do to a Pandas object.

```
(base) toth@toth-wl:~/Desktop$ python3 testpd.py
Major GPA
0  CSC  3.2
1  DSC  2.6
2  ECO  4.0
3  MAT  3.5
4  CSC  3.5
5  DSC  4.0
6  ECO  4.0
7  MAT  3.5
8  CSC  3.1
9  DSC  2.8

===

Major GPA
CSC    3.266667
DSC    3.133333
ECO    4.000000
MAT    3.500000
(base) toth@toth-wl:~/Desktop$
```

GroupBy

Can also group by one column and do aggregate op on another column

```
import seaborn as sns
import pandas as pd

planets = sns.load_dataset('planets')
print(planets)

print("\n===\n")

new_df = planets.groupby('year')['orbital_period'].median()
print(new_df)
```

```
(base) tothetoth-wl:~/Desktop$ python3 testpd.py
method number orbital_period mass distance year
0 Radial Velocity 1 269.300000 7.10 77.40 2006
1 Radial Velocity 1 874.774000 2.21 56.95 2008
2 Radial Velocity 1 763.000000 2.60 19.84 2011
3 Radial Velocity 1 326.030000 19.40 110.62 2007
4 Radial Velocity 1 516.220000 10.50 119.47 2009
... ..
1030 Transit 1 3.941507 NaN 172.00 2006
1031 Transit 1 2.615864 NaN 148.00 2007
1032 Transit 1 3.191524 NaN 174.00 2007
1033 Transit 1 4.125083 NaN 293.00 2008
1034 Transit 1 4.187757 NaN 260.00 2008

[1035 rows x 6 columns]

===

year
1989 83.888000
1992 45.901950
1994 98.211400
1995 4.230785
1996 65.669700
1997 39.845000
1998 58.112890
1999 256.780000
2000 179.665000
2001 462.790000
2002 505.549000
2003 441.470000
2004 35.103500
2005 68.630000
2006 233.150000
2007 311.665000
2008 226.930000
2009 464.700000
2010 16.278350
2011 20.219180
2012 11.738714
2013 24.806000
2014 12.815900
Name: orbital_period, dtype: float64
(base) tothetoth-wl:~/Desktop$
```

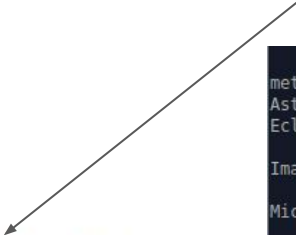
GroupBy

Can also group by multiple columns - just put them in a list

```
import seaborn as sns
import pandas as pd
```

```
planets = sns.load_dataset('planets')
```

```
new_df = planets.groupby(['method', 'number']).mean()
print(new_df)
```



	method	number	orbital_period	mass	distance	year
	Astrometry	1	631.180000	NaN	17.875000	2011.500000
	Eclipse Timing Variations	1	5821.166667	5.125000	NaN	2010.666667
		2	4216.883333	NaN	315.360000	2009.666667
	Imaging	1	140621.606250	NaN	71.683929	2009.205882
		4	73500.000000	NaN	39.940000	2008.500000
	Microlensing	1	3030.000000	NaN	4160.000000	2009.736842
		2	3462.500000	NaN	4080.000000	2010.000000
	Orbital Brightness Modulation	1	1.544929	NaN	NaN	2013.000000
		2	0.291496	NaN	1180.000000	2011.000000
	Pulsar Timing	1	18262.545353	NaN	1200.000000	2007.000000
		3	63.338433	NaN	NaN	1992.666667
	Pulsation Timing Variations	1	1170.000000	NaN	NaN	2007.000000
	Radial Velocity	1	814.143794	3.323939	60.648243	2007.082317
		2	959.022946	2.229547	49.316190	2008.219697
		3	769.421724	0.916872	30.302308	2008.886792
		4	695.769616	0.986492	9.930000	2003.823529
		5	1045.993508	1.166750	12.530000	2004.000000
		6	213.752357	0.038556	19.673333	2010.777778
	Transit	1	12.301023	1.470000	584.836460	2010.282927
		2	27.589652	NaN	924.142857	2012.375000
		3	22.187586	NaN	481.007143	2011.870968
		4	21.430775	NaN	NaN	2013.272727
		5	30.584464	NaN	320.333333	2012.080000
		6	40.513600	NaN	613.000000	2010.000000
		7	119.217898	NaN	780.000000	2013.000000
	Transit Timing Variations	2	79.783500	NaN	1104.333333	2012.000000
		3	NaN	NaN	NaN	2014.000000

Pivot Tables

Can do same stuff as groupby but often simpler syntax

```
import seaborn as sns
import pandas as pd

titanic = sns.load_dataset('titanic')

print(titanic.head())
```

```
(base) toth@toth-wl:~/Desktop$ python3 testpd.py
survived  pclass  sex    age  sibsp  parch  fare embarked  class  who  adult_male  deck  embark_town  alive  alone
0         0       3   male  22.0     1     0   7.2500     S   Third   man         True   NaN   Southampton  no    False
1         1       1  female  38.0     1     0  71.2833     C   First  woman        False   C     Cherbourg   yes    False
2         1       3  female  26.0     0     0   7.9250     S   Third  woman        False  NaN   Southampton  yes    True
3         1       1  female  35.0     1     0  53.1000     S   First  woman        False   C     Southampton  yes    False
4         0       3   male  35.0     0     0   8.0500     S   Third   man         True  NaN   Southampton  no     True
(base) toth@toth-wl:~/Desktop$
```

Pivot Tables

Can do same stuff as groupby but often simpler syntax

```
(base) toth@toth-wl:~/Desktop$ python3 testpd.py
  survived  pclass    sex  age  sibsp  parch    fare embarked  class  who  adult_male  deck  embark_town  alive  alone
0         0      3   male  22.0     1     0   7.2500         S  Third    man         True   NaN  Southampton    no  False
1         1      1  female  38.0     1     0  71.2833         C  First  woman        False    C    Cherbourg   yes  False
2         1      3  female  26.0     0     0   7.9250         S  Third  woman        False   NaN  Southampton   yes   True
3         1      1  female  35.0     1     0  53.1000         S  First  woman        False    C    Southampton   yes  False
4         0      3   male  35.0     0     0   8.0500         S  Third    man         True   NaN  Southampton    no   True

===

      survived
sex
female  0.742038
male    0.188908
(base) toth@toth-wl:~/Desktop$
```

```
import seaborn as sns
import pandas as pd

titanic = sns.load_dataset('titanic')

print(titanic.head())

print("\n===\n")

print(titanic.groupby('sex')[['survived']].mean())
```

What % of women survived?

What % of men survived?

Pivot Tables

```
(base) toth@toth-wl:~/Desktop$ python3 testpd.py
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True

```
====
```

sex	class	
female	First	0.968085
	Second	0.921053
	Third	0.500000
male	First	0.368852
	Second	0.157407
	Third	0.135447

Name: survived, dtype: float64

```
(base) toth@toth-wl:~/Desktop$
```

← This % of women traveling in First class survived...

```
import seaborn as sns
import pandas as pd

titanic = sns.load_dataset('titanic')

print(titanic.head())

print("\n===\n")

print(titanic.groupby(['sex', 'class'])['survived'].aggregate('mean'))
```

Pivot Tables

sex	class	
female	First	0.968085
	Second	0.921053
	Third	0.500000
male	First	0.368852
	Second	0.157407
	Third	0.135447

same data but reformatted

class	First	Second	Third
sex			
female	0.968085	0.921053	0.500000
male	0.368852	0.157407	0.135447

```
print(titanic.groupby(['sex', 'class'])['survived'].aggregate('mean'))
```

```
print(titanic.groupby(['sex', 'class'])['survived'].aggregate('mean').unstack())
```

```
print(titanic.pivot_table('survived', 'sex', 'class'))
```

rows

columns

Pivot Tables

Had this before:

class	First	Second	Third
sex			
female	0.968085	0.921053	0.500000
male	0.368852	0.157407	0.135447

```
print(titanic.pivot_table('survived', 'sex', 'class'))
```

Now get this

```
(base) toth@toth-wl:~/Desktop$ python3 testpd.py
class      First      Second      Third
sex  survived
female 0      25.666667  36.000000  23.818182
       1      34.939024  28.080882  19.329787
male  0      44.581967  33.369048  27.255814
       1      36.248000  16.022000  22.274211
(base) toth@toth-wl:~/Desktop$
```

```
print(titanic.pivot_table('age', ['sex', 'survived'], 'class'))
```

Can pass lists in place of rows or cols to make it multidimensional!

Avg age of people

Pandas Day 5 Lab Time!

Lab linked to Moodle

Pandas Day 6

Pandas Day 6 Topics

1. Retrieving a column
2. Transposing a DataFrame
3. Reordering columns
4. Mapping

Retrieving A Column

We got a row before, but how can we get columns?

Get a column as a series

- `x = df.col_name`
- `x = df['col_name']`

Can also pass a list to get multiple columns like `x = df[['col_name1', 'col_name2']]`

Retrieving A Column

```
import pandas as pd

# Build a DataFrame
df = pd.DataFrame({"Title":["Lightning", "The Stand", "Homeland"],\
                  "Author":["Dean Koontz", "Stephen King", "R.A. Salvatore"],\
                  "Pages":[384, 1142, 352]})

# Print the DataFrame
print(df)
print("\n===\n")

# Get the Author column one way
my_series = df.Author
print(my_series)
print("\n===\n")

# Get the Author column another way
my_series2 = df['Author']
print(my_series2)
print("\n===\n")
```

```
toth@toth-Latitude-5520 ~/Desktop
$ python3 demo.py
      Title      Author  Pages
0  Lightning    Dean Koontz   384
1   The Stand   Stephen King  1142
2   Homeland  R.A. Salvatore   352

===

0      Dean Koontz
1      Stephen King
2      R.A. Salvatore
Name: Author, dtype: object

===

0      Dean Koontz
1      Stephen King
2      R.A. Salvatore
Name: Author, dtype: object

===
```

Transposing A DataFrame

df.T

```
# Transpose the DataFrame
print(df)
print("\n===\n")
print(df.T)
```

```
      Title      Author  Pages
0  Lightning   Dean Koontz   384
1   The Stand  Stephen King  1142
2   Homeland  R.A. Salvatore   352
```

===

	0	1	2
Title	Lightning	The Stand	Homeland
Author	Dean Koontz	Stephen King	R.A. Salvatore
Pages	384	1142	352

Reordering Columns

```
import pandas as pd

# Build a DataFrame
df = pd.DataFrame({"Title":["Lightning", "The Stand", "Homeland"],\
                  "Author":["Dean Koontz", "Stephen King", "R.A. Salvatore"],\
                  "Pages":[384, 1142, 352]})

# Print the DataFrame
print(df)
print("\n===\n")

# Reorder columns (returns a new DataFrame)
df2 = df.iloc[:, [2,0,1]]
print(df2)
print("\n===\n")

# Reorder columns (returns a new DataFrame)
df3 = df2.loc[:, ["Author", "Title", "Pages"]]
print(df3)
```

```
toth@toth-Latitude-5520 ~/Desktop
$ python3 demo.py
```

	Title	Author	Pages
0	Lightning	Dean Koontz	384
1	The Stand	Stephen King	1142
2	Homeland	R.A. Salvatore	352

```
===
```

	Pages	Title	Author
0	384	Lightning	Dean Koontz
1	1142	The Stand	Stephen King
2	352	Homeland	R.A. Salvatore

```
===
```

	Author	Title	Pages
0	Dean Koontz	Lightning	384
1	Stephen King	The Stand	1142
2	R.A. Salvatore	Homeland	352

Mapping

Used to transform data, “mapping” it from one value to another

Concept:

Return a new series with the values from the old series having been adjusted as per a function

Syntax:

some_series.map(*some_function*)

where *some_function* has been defined

Mapping

```
import pandas as pd

# Our function for the mapping transformation
def fix_name_format(name):
    return name.lower()

# Build a DataFrame
df = pd.DataFrame({"Title":["Lightning", "The Stand", "Homeland"],\
                   "Author":["Dean Koontz", "Stephen King", "R.A. Salvatore"],\
                   "Pages":[384, 1142, 352]})

# Print the DataFrame
print(df)
print("\n==\n")

# Perform the mapping transformation
df['Author'] = df['Author'].map(fix_name_format)
print(df)
```

```
toth@toth-Latitude-5520 ~/Desktop
$ python3 demo.py
```

	Title	Author	Pages
0	Lightning	Dean Koontz	384
1	The Stand	Stephen King	1142
2	Homeland	R.A. Salvatore	352

```
===
```

	Title	Author	Pages
0	Lightning	dean koontz	384
1	The Stand	stephen king	1142
2	Homeland	r.a. salvatore	352

Pandas Project

Start the Pandas project linked to Moodle